Authentication research · workers-hono-svelte

Nov 5, 2026 · @Rob

Summary

Build authentication as a gate in the api Worker, keep sessions as rows in D1, hand-roll the primitives rather than adopt a framework, and treat a separate auth Worker as a deployment choice to make later, not now. The five provisional decisions all survive; three are amended, and the amendments (a `channel` gate before `authenticate`, processes as the layer that crosses areas, and `access.md` split into `requires` and `scope`) are the findings that change the story.

The recommended architecture, in chain order. Gates in the api: `requestId`, `bodyLimit`, `channel` (origin and content-type discipline for cookie-bearing unsafe requests), `authenticate` (one D1 statement joining `sessions` to `users`, run on the primary; an unknown cookie becomes `anonymous` with a clear-cookie hint, an unknown bearer token is refused, a disabled user is refused). The BFF sets a `__Host-` cookie on its own host and forwards the browser's request over the service binding; it decides nothing. `sessions.start` returns the token in the envelope and each delivery stores it its own way (cookie, CLI keyring). The access runner applies `requires` before validation; the operation passes a typed `scope` to the store; the store reports facts. Sign-up, disable and email verification are processes because they cross `users` and `auth`. Passwords hash with Argon2id via module-imported WASM on the Paid plan; passkeys use `@simplewebauthn/server`; Google OIDC and magic links are about a hundred lines each; admin can sit behind Cloudflare Access on its own hostname. Every number above has a budget test: one D1 round trip per gated request, at most two rows read, zero written on the steady path.

Three decisions the owner still has to make:

1. **Paid plan, and therefore passwords.** The Free plan's 10 ms CPU cannot hold an OWASP-compliant hash (production caps PBKDF2 at 100,000 iterations; confirmed open on workerd #1346 with no timeline). Either the lab runs on Paid ($5 a month, Argon2id at about 20 ms) or it offers passkeys, magic links and Google only. This decides whether email+password is in slice B at all.

2. **The email sender.** Cloudflare Email Service sends natively from a Worker binding since 16 Apr 2026, but it is still beta, Paid-only and quota-ramped for new accounts. Pick it or a third party (Resend, Postmark, SES); either way the sender is a port, so the choice is reversible. This decides what the first process (sign-up verification) talks to.
3. **Sliding idle window, or absolute only.** An idle window that extends on activity needs a throttled write outside the gate and a second timestamp; absolute-only sessions (sign in again after N hours) keep the gate read-only and the schema simpler. OWASP wants both for high-value apps. This decides the `sessions` schema before slice A writes it.

Not a decision yet: a separate auth Worker. It earns its keep only when a second consumer Worker appears, when sessions must live in a different database or jurisdiction, or when a Durable Object becomes the session write authority (adapter-cloudflare 7.x cannot export one from the SvelteKit Worker). None of those is true in slice A.

1. Where authentication runs

The api's gate should interpret the credential (option a), and the one fact that decides it is simple: a service binding forwards the browser's `Request` whole, cookies and `Origin` included, in-process and with no network hop. The BFF therefore needs no authority of its own; it forwards. Option b (the BFF decides, the api trusts a header) works only while the api is reachable by binding alone, and the provider CLI is already the second consumer that breaks it. Option d, a separate auth Worker, is a deployment decision, not a modelling one; the section below says when it earns its keep.

Criterion	a. api gate	b. BFF gate	c. both	d. auth Worker
One authority for identity	yes	only while the api is binding-only	no	yes (identity only)
How the actor reaches an operation	Hono context, set by the gate	`event.locals` → trusted header	duplicated	RPC `Result` → caller's context
Where CSRF is checked	api (cookie is forwarded)	BFF	both	BFF (api sees a non-ambient credential)

Criterion	a. api gate	b. BFF gate	c. both	d. auth Worker
Provider CLI	reuses the gate with a token file	must re-implement the gate	—	reuses `verify`
Public api hostname later	fine (csrf + bearer)	breaks the trusted header	—	fine
D1 round trips per page load	2	2	3	2 (lookup) / 1 (signed assertion)
Auth store down	every entry refuses	every entry refuses	every entry refuses	lookup: all refuse; assertion: only sign-in refuses
OpenAPI	one document	one	one	a second contract (RPC interface + contract test)
Local dev	2 Workers	2	2	3 Workers under one `wrangler dev -c` or Vite `auxiliaryWorkers`

CSRF follows the credential, not the network. The api needs CSRF protection exactly when it accepts a cookie as a credential, binding-only or not. If the BFF forwards the browser's cookie (a), a cross-site form POST to the BFF is a CSRF against the api, so either SvelteKit's origin check (form content types, POST/PUT/PATCH/DELETE, production only) or Hono's `csrf()` (checks `Origin` or `Sec-Fetch-Site`, same scope) must run; Hono's works unchanged behind a binding because the headers arrive intact. If the BFF converts the cookie into a header or RPC argument (b, d), the api receives nothing a browser could have attached and needs no check. A bearer token from the CLI is never ambient and needs none either. Hono 4.13.16 (1 Nov 2026) is a security release for `csrf()`.

Cookie scope. Cookies are scoped by registrable domain, not by Worker. A `__Host-` session cookie (the right choice: no `Domain`, `Path=/`, `Secure`) is locked to the BFF's hostname, which is exactly right when the api is only reached by binding. If the api ever

gets a sibling hostname, SvelteKit's server `fetch` does not forward a cookie to a sibling subdomain; `handleFetch` has to copy it, or better, send it as a bearer header. Different zones would force `SameSite=None` and are the wrong design for a BFF.

When a separate auth Worker (d) earns its separation on Cloudflare. When more than one consumer Worker needs the same gate and you want its code and D1 binding in one deployable; when sessions should live in a different D1 database or a Durable Object from business data (jurisdictions, backup cadence); when credentials code wants its own deploy cadence and blast radius; or when Cloudflare Access fronts admin and one place must validate `Cf-Access-Jwt-Assertion`. What it costs: identity keeps one authority but the Worker must stay at `issue / verify / revoke` and never grow role checks; an RPC-only surface has no OpenAPI document, so the address-and-check rule needs a TypeScript interface plus a contract test; each `verify` is a D1 read unless the assertion is self-verifying (then revocation lags by the assertion's TTL); and local dev becomes three configs under one `wrangler dev` (only the primary is exposed on `:8787`). One concrete pull toward d: `@sveltejs/adapter-cloudflare` 7.x says Durable Object and Workflow classes cannot be exported from the SvelteKit Worker, so a DO-backed session store would need its own Worker anyway.

Environment facts that frame everything (as of 5 Nov 2026): Hono stable 4.13.16, 5.0.0-rc.1 since 10 Oct; the svelte.dev docs now describe SvelteKit 3.x, where adapter-cloudflare 8.0.x removes `event.platform` in favour of `import { env } from 'cloudflare:workers'` and `csrf.checkOrigin` gives way to `csrf.trustedOrigins`; the lab's 2.x/adapter 7.x line keeps `event.platform.env`. Wrangler 4 multi-Worker dev is `wrangler dev -c primary.jsonc -c other.jsonc`; the old `--x-*` experimental flags are gone. `ctx.access` from Cloudflare Access is not propagated over service bindings or RPC, so Access can only ever gate the public hostname.

Round trips the budget test asserts (D1 or DO hops; binding hops are free):

Chain	a	b	c	d lookup	d signed
Page load: actor + one business read	2	2	3	2	1
api call from the browser via the BFF	2	2	3	2	1
Provider CLI operation	2	re-implements	—	2	2

D1 read replication (still beta, docs updated 10 Aug 2026) adds a wrinkle: a sign-out written to the primary must not be read back stale, so the gate's lookup should run `first-primary` or carry the session bookmark; replication is a later optimisation for read-

mostly business queries, not for the gate.

2. Session storage on Cloudflare

A D1 `sessions` table is the right store at both scales, and it costs about $0 a month at a few hundred thousand users because reads stay inside the 25 billion rows the Paid plan includes. Every alternative either cannot revoke (KV, a stateless cookie) or buys consistency D1 already has at higher cost and worse geography (Durable Objects). Scale A below is 300 users, about 20k gated requests a day; scale B is 300k users, about 20M a day; costs are incremental to the $5 Workers Paid plan, from the pricing pages as read on 5 Nov 2026.

Criterion	D1 table	Workers KV	DO per user	One coordinator DO	Signed stateless cookie	Short cookie + D1 re-mint
Consistency	strong on the primary	eventual, up to 60 s or more; negative lookups cached too	strong, single-threaded	strong	none to keep	bounded by cookie TTL
Immediate sign-out / disable	yes, one statement	no	yes, inside the object	yes	no	within TTL
Read latency	one hop to the primary's region	hot: edge; cold: central	one hop to the user's home colo, placed at first sign-in, never moves	one colo for the planet	zero	zero steady, one per TTL
TTL	`expires_at` column + cron sweep	native `expirationTtl`	one alarm per object	one alarm	`exp` claim	`exp` claim
Cost, scale A	$0	$0	$0 to $5	$0	$0	$0
Cost, scale B per month	about $0	about $440	about $90 plus $1.7k to $9k duration	about $90; peaks near the 1,000 req/s per-object limit	$0	about $0

Criterion	D1 table	Workers KV	DO per user	One coordinator DO	Signed stateless cookie	Short cookie + D1 re-mint
Budget test asserts	1 round trip, at most 2 rows read, 0 written	1 KV read	1 DO request	1 DO request	0	0 steady, 1 per TTL
Store down	all refuse	all refuse; stale cache can mask it	that user refuses	all refuse	cannot fail, cannot revoke	fail-open within TTL is a choice

D1, the provisional choice, holds. Without the Sessions API every query goes to one primary, so reads see writes. Keep replication off for the gate's lookup, or run it `first-primary`; the disable-user case has no shared bookmark between the admin's write and the victim's next request, so only the primary gives it immediate effect. Rows read count scanned rows, so the lookup must be an indexed point read (primary key on the id, index on `user_id` for the disable fan-out), and the session row should join the user row in one statement. The one cost trap is writing on every request: touching `last_seen_at` each time is about 600M writes a month at scale B (about $550); refreshing the sliding window only when it is older than, say, 15 minutes is about 30M, inside the 50M included. That is a design rule worth writing down: the gate reads, and writes only when the window needs extending. D1 returns `meta.rows_read` and `meta.rows_written` on every query, so the budget test asserts them directly without a mock. Since 11 Sep 2025 D1 retries read-only queries twice on retryable errors.

KV is a cache, not an authority. The docs are explicit: eventually consistent, usually immediate in the writing colo, up to 60 seconds or more elsewhere, and a cached "not found" delays visibility further. Sign-out and disable are deletes, so a revoked session keeps authenticating anywhere the old value is cached. It also bills every read, cache hits included, at roughly 500 times D1's per-row read price, and allows one write per second per key. It violates "stores report facts" by design: the fact may be a minute old.

A Durable Object per user is an anti-pattern for the gate and a reasonable authority for session mutations. For the read path: every request routes to the object's home colo, placed at the user's first sign-in and never relocated, so travelling users pay a cross-region hop on every call; duration is billed while the object stays awake (about 10 s after each request), which is where the $1.7k–9k range at scale B comes from, depending on how bursty traffic is; the first request after idle re-runs the constructor on the hot path; and single-threaded execution serialises an idempotent read for no benefit. Where it fits: issuing, revoking one, revoking all, and concurrent-session limits, the write-ordering

problems the KV docs themselves say to solve with a DO. A per-session object is strictly worse, since "disable user" then needs an index elsewhere. One coordinator object is fine at scale A and wrong at scale B (one colo for everyone, ~230 req/s average with peaks at the per-object limit). SQLite-backed DOs went GA on 7 Apr 2025 with a free tier; storage billing began 7 Jan 2026.

A stateless signed cookie cannot be ended, which contradicts the two requirements already written (sign-out ends the row, disable ends all rows) and OWASP's rule that logout invalidates server-side. If it is ever used, a plain HMAC-SHA-256 over `session_id|user_id|exp` with no parsed algorithm, never a JWT, for a first-party cookie.

The one hybrid worth keeping in reserve: a short-lived HMAC cookie (about 5 minutes) carrying `session_id`, `user_id`, `minted_at`; the gate skips D1 while the cookie is fresh and re-reads and re-mints when it is not. Steady-state store reads become zero, revocation lags by at most the TTL, and one operation still owns the decision. This is the first move if latency for users far from the primary ever fails its test, not KV and not DOs.

Session id and cookie, per OWASP and rfc6265bis-22 (Dec 2025). 256 bits from `crypto.getRandomValues`, base64url; store only a SHA-256 of it so a database dump cannot be replayed. Rotate on sign-in and on any privilege change, destroy the old id, never accept an id the server did not issue. Idle and absolute timeouts enforced server-side; for a bookkeeping product the high-value end of OWASP's examples (short idle, 4–8 h absolute) is the honest choice. Cookie: `__Host-sid=…; Secure; HttpOnly; SameSite=Lax; Path=/`, no `Domain`. `Clear-Site-Data` on sign-out; `Cache-Control: no-store` on sensitive replies. A CLI token is a separate opaque token in the same table with a `kind` column and the same revocation path.

Rob's primary belongs in `wnam`; a same-region hop is single-digit to low-tens of milliseconds and a trans-Atlantic one adds roughly 80–120 ms (planning figures, not from a Cloudflare page).

3. Identity methods and their libraries

The lightweight-library tier this design would have reached for no longer exists: Lucia was deprecated in March 2025, Arctic and the `@oslojs/*` packages on 29 Jul 2026, all by their author, who now publishes single-file replacement code and the Auth Book instead; Auth.js was folded into Better Auth on 22 Sep 2025 and receives security patches only; Better Auth became a Vercel company on 7 Jul 2026. The field has consolidated onto

Better Auth as the framework and copy-the-primitives as the alternative, and for a design where the api owns the decision, copy-the-primitives is the better fit. The one library worth adopting outright is `@simplewebauthn/server`.

Method	What it needs from the platform	Library today	States the BFF renders	Threats specific to it
Email + password	a pepper in a Workers secret; CPU for hashing (section below); email sending; a throttle store	`@phi-ag/argon2` 0.6.2 (21 Oct 2026); the rest is about 200 lines of your own	unverified, wrong credentials (one generic message), disabled, reset sent (always), reset invalid or expired, password changed and other sessions ended	stuffing and enumeration; offline cracking if D1 leaks (hence the pepper); timing leaks (never await the email send); bcrypt's 72-byte cap if ever used
Passkeys / WebAuthn	`rpID` = the registrable domain shared by BFF and api; the allowed origins; a per-ceremony challenge row with a short TTL; a credentials table (id, public key, counter, transports, backed-up, device type, aaguid)	`@simplewebauthn/server` 14.0.3 (25 Sep 2026; Node 22+; Workers "periodically tested, unofficially supported" since v8) + `/browser` 14.0.0	no passkey yet, conditional UI autofill (`autocomplete="username webauthn"`), discoverable vs not, cancelled, counter regression	rp id or origin mismatch; challenge replay unless single-use; cloned authenticator via counter; phishing resistance holds only with a strict `expectedOrigin`
Google OIDC	client id and secret as secrets; an exactly registered redirect URI (scheme, case, trailing slash); state, nonce and PKCE verifier stored per attempt; JWKS from the discovery document	no maintained client worth adopting (Arctic is deprecated); about 100 lines against the discovery document, with `hono/jwt` `verify()` or `jose` for the id token	redirect, callback success, denied, state mismatch, link prompt for an existing local user, email unverified at the provider	login CSRF (state); code injection (PKCE S256); replay (nonce); audience confusion; link by email only when `email_verified` is true, and key on `sub` because Google says the email can change
Magic links	email sending; a token table with the token's hash, expiry (about 5 minutes), single-use consume; a BFF landing route that POSTs the token to the api	your code	sent (always the same message), expired, already used, signed in on a new device	token leaks via Referer, logs and mail scanners (store hashed, consume only on an explicit POST, never on a GET that a mail scanner prefetches); sign-up by default creating accounts for typos

Method	What it needs from the platform	Library today	States the BFF renders	Threats specific to it
Admin via Cloudflare Access	an Access application on the admin hostname, its AUD tag, the team's `/cdn-cgi/access/certs`; validate `Cf-Access-Jwt-Assertion` (the cookie is not guaranteed); keys rotate every 6 weeks with a 7-day overlap, so fetch, never pin	`hono/jwt` `verify()` with RS256, `aud` and `iss` checks	denial is rendered by Cloudflare; inside the app the actor is the Access identity mapped to a `users` row or refused	trusting the header without verifying the signature; AUD mismatch admitting another app's token; the admin Worker reachable on a hostname Access does not front

Every method asks the BFF for the same thing: one place that turns an api decision into a page state. They differ only in the verb (`sign_in` , `link_account` , `verify_email` , `consume_magic_link` , `finish_passkey`) and the states above; hashing, JWKS and challenge logic never reach the BFF.

Library	Version, last release	On Workers	Owns routes	Owns cookies	Owns schema	Fits api-owned decisions
Better Auth	1.7.18, 5 Nov 2026 (1.8 in beta)	yes with `nodejs_compat`; D1 via its Kysely adapter; migrations cannot run from the CLI against D1	wants `app.all("/api/auth/*", auth.handler)`, but `auth.api.signInEmail({ body, headers })` is a plain function and `asResponse` / `returnHeaders` hand you the `Set-Cookie`	yes: `better-auth.session_token` (opaque, signed), `session_data`; renamable	yes: `user`, `session`, `account`, `verification`; columns fixed, names remappable, extras via `additionalFields`	partially: sessions are already server-side rows, which matches the provisional decision; but it throws `APIError`, merges credentials and OAuth links in one `account` table, and defaults to scrypt 16384/16/1. Usable as a provider behind the operation layer; costly if you fight its schema
Auth.js / `@auth/sveltekit`	1.11.3, 20 Jul 2026	yes with `AUTH_TRUST_HOST`	yes: its `handle` hook receives all requests and owns `/auth/*`	yes	yes via adapters	no: it makes the BFF the authority; maintenance mode since Sep 2025; its own docs still call the SvelteKit package experimental

Library	Version, last release	On Workers	Owns routes	Owns cookies	Owns schema	Fits api-owned decisions
Lucia	3.2.2, Oct 2024, deprecated Mar 2025	n/a				now a guide with `auth_session.ts` as a single-file replacement and the Auth Book (sessions, Argon2, passkeys, email codes, CSRF): the closest thing to a blessed design for an opaque session id in a table, and a good fit precisely because it is not a library
Arctic, Oslo	deprecated 29 Jul 2026 (only `@oslojs/encoding` kept)					the author's own verdict: OAuth 2.0 is a poor layer to abstract into a library; copy the pattern
`@simplewebauthn/server`	14.0.3, 25 Sep 2026	WebCrypto-based since v8; no Node-native modules	no, pure functions	no	no, you store the credential	yes, cleanly: `expectedOrigin` and `expectedRPID` accept arrays; returns `{ verified }` for mismatches and throws only on malformed input (wrap it)
OpenAuth (`@openauthjs/openauth`)	0.4.3, 4 Mar 2025, still beta, no release in 20 months	built on Hono; issuer on Workers; KV storage	yes, entirely: it is a separate issuer Worker with OAuth2/OIDC endpoints and a login UI	yes	no user table by design	this is the second-auth-api architecture, with JWT access tokens rather than D1 rows; stale cadence; reasonable only for standards-based SSO across many clients
`hono/jwt`, `hono/cookie`	Hono 4.13.16, 1 Nov 2026	native	the JWT middleware reads a header or cookie and throws `HTTPException`; the helpers (`sign`, `verify`, `decode`) own nothing	`hono/cookie` owns nothing and enforces rfc6265bis (`__Host-` rules, 400-day cap)	no	yes: use the helpers inside the gate, never the middleware
Cloudflare first-party	Access (admins); `@cloudflare/workers-oauth-provider` (an OAuth 2.1 provider for MCP clients, not an identity provider); Email Service (beta); Turnstile		Access owns the login UI for a hostname	Access sets `CF_Authorization`	none	there is no first-party end-user authentication product; Cloudflare's own tutorials hand-roll it

Password hashing is the real platform constraint. The Free plan allows 10 ms CPU per request, Paid 30 s by default (raisable to 5 min). Production Workers cap WebCrypto PBKDF2 at 100,000 iterations (workerd issue #1346; the open-source runtime no longer enforces it, so `wrangler dev` runs 600,000 without error and local tests will not catch it), while OWASP's floor for PBKDF2-SHA256 is 600,000. `node:crypto` exposes `pbkdf2` and `scrypt` but not `argon2`. Measured on 5 Nov 2026 in local workerd (wall time, 2 vCPU, best of three): PBKDF2 100k 12 ms; scrypt at Better Auth's default 52 ms, at OWASP's N=2^17 230 ms; pure-JS Argon2id (`@noble/hashes`) 142 ms; `hash-wasm` fails outright ("Wasm code generation disallowed by embedder", because it compiles base64-embedded WASM at runtime); `@phi-ag/argon2` with the `.wasm` imported as a module 21 ms at OWASP's m=19456 t=2 p=1, 19 ms at 12288/3/1, 26 ms at 47104/1/1.

Recommendation: Paid plan, Argon2id via `@phi-ag/argon2` at m=19456 KiB, t=2, p=1, pepper by HMAC-SHA256 from a Workers secret before hashing, PHC strings stored with their parameters, re-hash on sign-in when parameters change. The package's `tryHash` / `tryVerify` return `{ success, error }`, which is errors as values. The hashing lives in the `auth` area's credential store, which reports verified, not verified, or needs-rehash, and the operation decides. On the Free plan nothing OWASP-compliant fits in 10 ms; the honest option is to not offer passwords there (passkeys, magic links and Google only). Caveat: these are wall-clock figures in local workerd; production CPU metering is opaque, and a community template reports Better Auth on Pages hitting CPU limits, solved by a separate hasher Worker over a service binding.

4. The Cloudflare product map

Four products carry the flow (Workers, service bindings, D1, Turnstile); three are optional helpers (Queues, the Rate Limiting binding, Secrets Store); two change a prior assumption: Cloudflare can now send email natively (Email Service, public beta since 16 Apr 2026, Paid plan only), and Cloudflare Access can gate individual Workers (14 Aug 2026) but its identity never crosses a service binding. Nothing here is Enterprise-only except WAF rate-limit keying on headers or cookies and Turnstile's extras. Status and limits as read on 5 Nov 2026.

Product	Role in the flow	Status	Limits and pricing	Verdict
Workers	gate, operations, BFF	GA	Free: 100k requests/day, 10 ms CPU; Paid: $5/mo, 10M requests and 30M CPU-ms included (page updated 2 Oct 2026)	baseline
Service bindings (HTTP)	BFF to api, forwarding the browser `Request`	GA	in-process, 32 invocations per chain; WAF, rate limiting and Access are not in the path	use for option a
RPC `WorkerEntrypoint`	typed `verify` / `issue`; pass a `Request` or an actor	GA (compat date 2024-04-03 or later)	32 MiB per message; named entrypoints per caller role; object-capability model for Workers that do not trust each other	use for b or d; `Result` values survive structured clone
D1	sessions and users tables	GA; read replication and the Sessions API still beta (since 10 Apr 2025); `us` jurisdiction added	Free: 5M rows read and 100k written per day, 500 MB, 50 queries per invocation; Paid: 10 GB per database, 1,000 queries per invocation, 25B rows read included; 100 bound parameters; 30 s statement timeout (limits updated 30 Oct 2026)	yes; the 50-query Free cap belongs in the chain test
KV	session cache only	GA; KV Instant in private beta (1 Oct 2026)	eventual, 60 s or more; one write per second per key; Instant: 1 MB per namespace, $0.10 per write	never authoritative for sessions
Durable Objects (SQLite)	strongly consistent write authority for sessions; strict global throttles	GA; free plan since 7 Apr 2025; storage billed from Jan 2026	Free: 100k requests/day, 5 GB; Paid: 1M requests included, then $0.15/M, $0.20/GB-month (updated 18 Oct 2026); cannot be exported from the SvelteKit Worker under adapter-cloudflare 7.x	optional; not on the gate's read path
Queues	async email and audit after sign-in	GA, on Free	Free: 10k operations/day, 24 h retention; Paid: $0.40/M	yes for side effects

Product	Role in the flow	Status	Limits and pricing	Verdict
Workflows	multi-step verification with retries and timeouts	GA; per-step billing from 10 Aug 2026	Free: 3,000 steps/day; Paid: 500k steps/month included; 25,000 steps per instance	optional; overkill for a two-step verification
Email Service, sending	verification and reset mail through a native `send_email` binding	public beta 16 Apr 2026; SMTP beta 8 Jun 2026	Paid only; 3,000/month included, $0.35 per 1,000; 50 recipients; 25 MiB; a rising daily quota for new accounts; Cloudflare DNS required	a third-party sender is no longer unavoidable, but it is beta and Paid-only: keep the sender behind a port so Resend, Postmark or SES stay swappable
Email Routing and Email Workers	receiving (bounces, reply-to-verify)	GA, Free	200 rules per domain	not needed for auth
Turnstile	bot check on sign-up and sign-in, verified from the entry point	GA	Free: 20 widgets, 10 hostnames each; ephemeral ids and off-label use are Enterprise	yes, the free tier suffices
Rate Limiting binding	per-email or per-account throttle on sign-in	not marked beta (Wrangler 4.36 or later)	per-location and approximate; 10 s or 60 s periods; not available as a remote binding in dev	yes for cheap throttling; a DO for strict limits
WAF rate limiting	IP-based protection on the BFF hostname	GA	Free: 1 rule, IP only, 10 s window; keying on headers or cookies is Enterprise Advanced (updated 25 Aug 2026)	one free rule on the sign-in path; cannot key on a session
Cloudflare Access	admin gate on a hostname or path; worker-level protection	worker-level Access 14 Aug 2026 (docs updated 2 Nov 2026)	Zero Trust Free up to 50 users, $7 per user beyond; path-scoped applications; `ctx.access` is not propagated over bindings or RPC; no WebSockets under worker-level policies	a good admin gate and a second authentication authority: declare it in the spec as the gate for the admin hostname only

Product	Role in the flow	Status	Limits and pricing	Verdict
Secrets Store	signing keys, email API keys	open beta (page updated 14 Aug 2026); not on the China network	`secrets_store_secrets` binding, `await env.X.get()`; no local access to production secrets	fine; plain Worker secrets are equivalent for one app
Remote bindings in dev	real D1 and KV from `wrangler dev`	GA 16 Sep 2025 (Wrangler 4.37 or later)	not for DOs, Workflows, secrets or the rate limiter	useful against staging data
Multi-Worker dev	BFF, api and a possible auth Worker locally	current (page updated 25 Jun 2026)	`wrangler dev -c a.jsonc -c b.jsonc` or Vite `auxiliaryWorkers`; only the primary is exposed	required for option d; recommended even for a

Region notes: D1 replicas are created automatically in ENAM, WNAM, WEUR, EEUR, APAC and OC; D1, KV and Workflows accept `eu`, `us` and `fedramp` jurisdictions; Secrets Store is unavailable on the JD Cloud China network. Unverified by the research: the Zero Trust plans page was blocked (figures taken from the SASE plan page) and the Secrets Store limits page failed to load. A later pass confirmed that `workers.dev` and `pages.dev` are on the Public Suffix List (section 5).

5. Threats and where each answer lives

Thirteen threats, and the answer to each has one home in the chain. Three facts verified in this pass change earlier assumptions: Hono 4.13.16's `csrf()` fix closes a Chromium `<a ping>` bypass, the third bypass of that middleware, so a content-type allow-list is a brittle primitive that needs a second check; `Clear-Site-Data` is blocked in Chromium since April 2025 (Firefox 138+ and Safari 17+ honour it), so it is a courtesy and server-side invalidation is the control; and `workers.dev` and `pages.dev` are on the Public Suffix List, so two Workers on `*.workers.dev` are cross-site to each other, which isolates them and also kills any plan where the browser talks to both hosts. ASVS ids are 5.0.

Threat	Mitigation	Source	Where it lives
CSRF on the cookie-authenticated api	two independent checks: `Sec-Fetch-Site` in {same-origin, none} with `Origin` as fallback, refusing when both are absent on an unsafe method (OWASP: a fallback to origin headers is mandatory for Fetch Metadata; missing headers are not evidence of same-origin); plus content-type discipline (JSON endpoints accept only `application/json`, never `text/plain`). The CORS preflight protects page fetches only if the api never emits a permissive `Access-Control-Allow-Origin`	OWASP CSRF; ASVS 3.5.1–3.5.3	BFF for form actions (SvelteKit's own check: form content types, unsafe methods, production only) and an api stage before the gate, since the api is what accepts the cookie; not the gate itself
SameSite=Lax top-level POST gap	set `SameSite=Lax` explicitly: the two-minute "Lax-allowing-unsafe" window applies only to cookies with no attribute (rfc6265bis §5.6.7.2); no state change on GET (SvelteKit actions are POST only); the check above covers cross-site top-level POST	rfc6265bis-22; OWASP CSRF	BFF (the attribute); a design rule for operations

Threat	Mitigation	Source	Where it lives
Session fixation, missing rotation	mint a new row on every sign-in and re-authentication, end the old one; ids are server-minted only, so classic fixation is structurally impossible, but rotation on privilege change is still required	OWASP Session Mgmt; ASVS 7.2.4	operation (`signIn` , `reauthenticate`); the BFF replaces the cookie
Cookie scope across two Workers	the browser sees one host, the BFF; `__Host-` (Secure, Path=/, no Domain); the api reachable only by binding, with `workers_dev = false` in config because a dashboard-only disable is re-enabled on the next deploy	ASVS 3.3.1, 3.3.3, 3.3.4; Cloudflare docs	BFF (cookie), platform (no public route), gate (reads only `Cookie` or `Authorization`)
Open redirect in OAuth	exact-string `redirect_uri` match; `state` as a one-time token or PKCE for login CSRF; an issuer check if ever more than one provider	RFC 9700 §2.1, §4.1.3, §4.4, §4.7.1	operation (`oauthStart` writes state and verifier; `oauthCallback` consumes them)
Open redirect in post-login `returnTo`	accept only a same-origin path: `pathname` starting with `/` and not `//` , no scheme, host or userinfo; or map to a server-side id; never prefix-match	OWASP Unvalidated Redirects	BFF only; the api never sees it

Threat	Mitigation	Source	Where it lives
Enumeration on sign-up, sign-in, reset, magic link	uniform body, status and timing: sign-up always says "a link has been emailed" and sends a different mail to an existing owner; sign-in runs the hash against a fixed dummy when the user is missing; reset and magic link always 200 and enqueue mail only if the account exists	OWASP Authentication, Forgot Password; ASVS 6.3.8 (L3)	operation (owns the response shape and the dummy-hash branch); process for sign-up
Credential stuffing, brute force	layered: a per-account failure counter in D1 with exponential delay and temporary lock (never permanent, which is a denial-of-service lever); Turnstile on sign-in, sign-up and reset, verified server-side (token 300 s, single use); the Rate Limiting binding keyed on account, not IP (per-location, approximate, 10 s or 60 s periods); one WAF rule on the sign-in path as a per-colo IP ceiling; a breached-password check at set time	OWASP Credential Stuffing, Authentication; ASVS 6.3.1, 6.1.1, 6.2.12	platform for Turnstile, the binding and WAF; operation for the account counter and the breach check; the BFF validates the Turnstile token before calling the api

Threat	Mitigation	Source	Where it lives
Verification, reset and magic-link tokens	CSPRNG, 256 bits, stored as SHA-256, bound to user and purpose, single use (`consumed_at`), expiry of 10 minutes or less for out-of-band requests, consumed only on an explicit POST (the GET renders a confirm page, which also defeats mail scanners), `Referrer-Policy: no-referrer` on that page, no auto sign-in after reset	OWASP Forgot Password; ASVS 6.5.1, 6.5.3, 6.5.5, 6.6.2	operation (`consumeVerification`); store (`UPDATE … WHERE hash = ? AND consumed_at IS NULL AND expires_at > ?` returning the change count); BFF (confirm page)
Sign-out semantics	this device ends one row; all devices ends every row for the user and requires re-authentication; emit an expiring `Set-Cookie` and `Clear-Site-Data` as a courtesy; users can list and end their sessions; admins can end anyone's	OWASP Session Mgmt; ASVS 7.4.1, 7.4.5, 7.5.2	operation (`signOut`, `endAllSessions`); BFF (headers)
Password change or reset leaving other sessions alive	change requires the current password; after change or reset rotate the current session and offer to end the others; notify the user; invalidate outstanding reset links	ASVS 6.2.3, 7.4.3, 6.3.7; OWASP Forgot Password	process (`changePassword`: verify, rewrite credential, rotate, optionally end others)

Threat	Mitigation	Source	Where it lives
Clickjacking and headers	`Content-Security-Policy: frame-ancestors 'none'` on every HTML response (X-Frame-Options is obsolete per ASVS 3.4.6); HSTS of a year or more with `includeSubDomains`; `nosniff`; Referrer-Policy; COOP same-origin; CSP with `object-src 'none'; base-uri 'none'` and nonces (SvelteKit `kit.csp` with `mode: 'auto'` does nonces for SSR and hashes for prerender; `frame-ancestors` is ignored in a `<meta>` tag, so it goes in the header from `handle`). Hono `secureHeaders()` sets X-Frame-Options, COOP, CORP, HSTS at 180 days and no CSP by default	OWASP Clickjacking; ASVS 3.4.1, 3.4.3–3.4.8	BFF `handle` for everything the browser renders; api `secureHeaders()` as belt and braces for JSON, HSTS raised to a year

Threat	Mitigation	Source	Where it lives
Subdomain takeover, cookie tossing	`__Host-` denies Domain-scoped cookies, so a sibling subdomain cannot plant one the BFF accepts; `workers.dev` being a public suffix makes preview Workers cross-site anyway; clean DNS; CORS allow-list as exact strings, never a subdomain pattern; the gate reads only the `__Host-` name	rfc6265bis §4.1.3.2; ASVS 3.3.3, 3.5.4; OWASP WSTG 4.2.10	BFF (prefix), platform (DNS hygiene, Access on preview URLs), gate

Two notes. The CSRF row is the one place the gate's two-things contract is under pressure: the check is not about who the actor is but whether the credential channel can be trusted for this request, so it belongs in a named stage ahead of the gate (section 6 takes this up). And the Rate Limiting binding is a cost shield, not a correctness control; the per-account counter in D1 is the control, and a proxy network defeats any per-colo IP ceiling.

6. Where the provisional decisions are wrong

None of the five is overturned; three are amended in ways that matter, and the amendments are the real output of this research. The pattern across them is the same: each decision was stated as "once" when the mechanism is actually two things, and naming the second thing is what keeps the story honest.

0.1 Authentication as a gate, authorization never in the gate: amend. The strongest case against is that "exactly two things" is already false once CSRF exists. A gate that reads a cookie must decide, per request, whether this cookie may be trusted at all, and OWASP treats origin verification as a mandatory part of trusting a cookie on an unsafe method. Left implicit, nobody owns the content-type allow-list, which is exactly how Hono's three `csrf()` bypasses happened. The fix is not a new abstraction: the origin and content-type check is itself a gate by the definition (it refuses in the envelope and attaches nothing,

like the body limit), so the ordered list grows by one, `requestId`, `bodyLimit`, `channel`, `authenticate`, and each gate keeps its two-things contract. Second, an anonymous-admitting gate is default-allow unless the runner is default-deny: an operation with no row in `access.md` must fail generation, not run for anonymous. Third, "applied once by a shared runner" is true for the role column and false for ownership, which cannot be known until the id is parsed and the row located; the role check runs before validation (and gives 403-before-400 for free), the scope is handed to the operation and enforced by the store, and the story should say two mechanisms. Last, refusing a bad cookie is the wrong default: a user who signs out in one tab and submits a form from another 200 ms later would see an error envelope instead of the sign-in page. An unknown or unparseable cookie becomes `anonymous` with a clear-cookie hint for the BFF; an unknown bearer token from the CLI is refused with 401, because the client asserted a credential and it is wrong; a live session for a disabled user is refused with `account_disabled`.

0.2 Server-side sessions in D1, `__Host-` cookie set by the BFF: keep, amend the minting. The decision never said who mints or how the raw id crosses the binding. If the api writes `Set-Cookie` and the BFF forwards the response, the cookie works (the browser scopes it to the BFF host) but the api now knows cookie names and attributes, and a CLI calling the same operation receives a `Set-Cookie` it must ignore. So `sessions.start` returns `{ token, expiresAt }` in the result envelope; the BFF delivery turns it into the cookie and the CLI into a keyring entry. The raw token lives in memory in two Workers for one request and at rest only as a SHA-256 (fast hashing is enough for 256 bits of entropy). Three more conditions: a `channel` column (`cookie` or `bearer`) with its own idle and absolute rules, and a bearer token never accepted from a `Cookie` header or the reverse; the gate's read runs on the primary (a replica with `first-unconstrained` can be arbitrarily stale, so a revoked session would read as live); the sliding idle window is extended at most once per N minutes and outside the gate, after the operation, via `waitUntil`, so the gate reads and never writes. And `workers_dev = false` in the api's config, since a dashboard-only disable comes back on the next deploy.

0.3 `users` and `auth` as two areas, `auth` may read `users`: amend the layering rule. The cycle appears immediately. Sign-up creates a user (a `users` authority) and then a credential and a verification (`auth`), so "auth may read users" must mean "auth may call users operations", and if `auth` wrote the `users` table through its own store there would be two authorities for user rows. Worse, `users.disable` is a users decision whose required effect, ending every session (ASVS 7.4.2), is an auth write that `users` may not reach. The resolution is already in the story's chain: a process is the stage above operations, and a process may compose operations from several areas while areas never

import each other. `signUp` and `disableUser` are processes. For correctness independent of step ordering, the gate's session lookup joins `users.disabled` in the one statement it already makes, so a disabled user is refused even if the row-ending step lagged or failed: the join is the control, ending the rows is hygiene. That also answers where `disabled` is enforced (the gate) and where `role` lives (`users`; it is a fact about the person, read by `auth` when it builds the actor). OAuth account links belong in `auth`, keyed on provider and `sub`; `email_verified_at` is a `users` fact written by the `verifyEmail` process after `auth.consumeVerification` succeeds, one more process that spans both.

0.4 Ownership by the scoped query, wrong owner reads as `not_found`: keep, with two conditions. The case against: a `WHERE user_id = ?` baked into the store is a decision; admin reads need a different query; the first shared resource (a team, a client's project) makes `user_id` the wrong fact; and `not_found` for an existing row can collide with real 404 handling in the BFF. It holds because the operation makes the decision when it chooses the scope it hands the store, `{ ownerId }`, `{ any: true }` for admin, later `{ memberOf }`; the store reports "no row for this scope" and the operation names it `not_found`. Timing is uniform because the query shape is identical for missing and foreign rows, and list endpoints scoped the same way cannot enumerate. OWASP has no 403-versus-404 rule, so `not_found` is legitimate; the one case it must not cover is "yours but refused by state", which is `forbidden` with a reason. Conditions: writes are scoped too (`UPDATE … WHERE id = ? AND user_id = ?` returning the change count, never read-then-write), and the scope is a typed input the operation passes, with no default scope in the store, which keeps default-deny.

0.5 `access.md` as a static operation × role × owns table: amend the columns. A static table can say who may attempt an operation; it cannot carry state-dependent conditions (verified email, a locked resource, an expired trial) or relationships beyond ownership without growing columns forever or lying by omission (ASVS 8.1.2 expects documented rules that depend on state). The columns become `requires` in {anonymous, member, verified_member, admin}, an actor precondition the runner applies before validation, and `scope` in {own, any, n/a}, handed to the operation as the store scope. `verified_member` earns a place because it is an actor fact from the gate's join, not a resource fact. Anything state-dependent is declared in the operation's own use case, enforced in the body after the row is loaded, and returns `forbidden` with a reason. What the check can verify: statically, every operation has exactly one row, every row names a real operation, cells use the closed vocabulary, and the generated `access.ts` matches the table; dynamically and cheaply, each operation called as each actor kind with an empty body asserts the runner's

verdict (401, 403, or "reached validation"), which is the budget-test style applied to access. What it cannot verify is that `own` is enforced, since that lives in the store scope; that needs a second check, a lint that every query against an owned table takes a scope parameter.

Sources

All pages opened on 5 Nov 2026; a page's own last-updated date is given where it shows one. Four load-bearing claims were re-opened in a second pass by the editor: Email Sending public beta (changelog entry 16 Apr 2026, no GA entry), the production PBKDF2 cap (workerd issue #1346, open, staff: no ETA), adapter-cloudflare 8.0.0 removing `event.platform` and requiring SvelteKit 3 (CHANGELOG, latest 8.0.2; 7.x latest 7.2.9), and Better Auth joining Vercel (post dated 7 Jul 2026). Versions as of the research date: Hono 4.13.16, 5.0.0-rc.1; Wrangler 4.158.0 used for local measurements; Better Auth 1.7.18; `@simplewebauthn/server` 14.0.3; `@phi-ag/argon2` 0.6.2; `@auth/sveltekit` 1.11.3; `@openauthjs/openauth` 0.4.3.

Cloudflare

- Service bindings overview, HTTP, RPC, Workers RPC and its security model
- Developing with multiple Workers (25 Jun 2026); remote bindings GA (16 Sep 2025); Wrangler commands
- Routes; workers.dev subdomain; Smart Placement (12 Oct 2026)
- Workers pricing (2 Oct 2026); Workers limits; Web Crypto; node:crypto support; workerd issue #1346 and workerd crypto impl
- D1: pricing, limits (30 Oct 2026), read replication (10 Aug 2026), data location, D1Database API, release notes, read replication beta (10 Apr 2025)
- KV: how KV works, read, write, pricing, limits, performance changelog (22 Aug 2025), Workers KV Instant (1 Oct 2026)
- Durable Objects: pricing, limits, lifecycle (30 Oct 2026), alarms, data location, FAQ, SQLite GA and free tier (7 Apr 2025)
- Queues pricing; Workflows changelog
- Email Service: changelog (public beta 16 Apr 2026), send from Workers, limits, pricing; Email Workers
- Turnstile, plans, server-side validation; Rate Limiting binding (21 Oct 2026); WAF rate limiting (25 Aug 2026) and request-rate calculation

- Access: Workers and Access (2 Nov 2026), validating JWTs, SASE plans; Secrets Store (14 Aug 2026) and Workers integration; workers-oauth-provider

Frameworks and libraries

- Hono: releases and v4.13.16, csrf, secure headers, cookie helper, JWT middleware and helper; CVE-2024-43787
- SvelteKit: hooks, configuration, form actions, load, adapter-cloudflare and its CHANGELOG, kit releases
- Better Auth: docs on GitHub main, server API, Auth.js joins Better Auth (22 Sep 2025), Better Auth joins Vercel (7 Jul 2026), better-auth-cloudflare; next-auth README; Auth.js SvelteKit
- Lucia and repo; Deprecating Lucia, Arctic, Oslo (29 Jul 2026); Auth Book; Arctic
- SimpleWebAuthn server and releases; OpenAuth; phi-ag/argon2; cloudflare-auth-hasher-template; Hashing passwords on Workers (21 Feb 2024); native email+password on Workers (10 Sep 2026)
- Local benchmarks: Node 22.23.4 and wrangler 4.158.0 (workerd), 5 Nov 2026, wall-clock, 2 vCPU, best of three

Standards and guidance

- OWASP cheat sheets: Session Management, Password Storage, CSRF Prevention, Authentication, Forgot Password, Credential Stuffing, Clickjacking, Authorization, Unvalidated Redirects; WSTG subdomain takeover
- OWASP ASVS 5.0: V3 Web Frontend, V6 Authentication, V7 Session Management, V8 Authorization
- draft-ietf-httpbis-rfc6265bis-22 (1 Dec 2025); RFC 9700 OAuth 2.0 Security BCP; W3C Fetch Metadata; W3C WebAuthn Level 3; Related Origin Requests; Google OpenID Connect
- Clear-Site-Data on MDN and browser support; Public Suffix List

Not retrievable during the research: MDN Set-Cookie, the OWASP JWT cheat sheet, the Zero Trust plans page, the Secrets Store limits page, some better-auth.com docs pages (read from the GitHub source instead), and rfc6265bis §8. Unverified: whether the PBKDF2 cap has been raised since the issue's last staff comment; the production scrypt cost cap; CPU-metered (rather than wall-clock) cost of the WASM hashing runs.